Efficient LLM Inference for Mathematical Reasoning under a Fixed Budget
Project Proposal for NLP Course, Winter 2026/27

Michał Kukla (submitted by)
311134
michal.kukla2.stud@pw.edu.pl

Katsiaryna Bokhan
329903
katsiaryna.bokhan.stud@pw.edu.pl

Sebastian Pergała
327301
sebastian.pergala.stud@pw.edu.pl

Norbert Frydrysiak
327273
norbert.frydrysiak.stud@pw.edu.pl

supervisor: Anna Wróblewska
Warsaw University of Technology
anna.wroblewska1@pw.edu.pl

Template: Anna Wróblewska (Warsaw University of Technology), adapted from the ACL 2015 conference formatting template.

Abstract

Running open-weight LLMs on complex mathematical reasoning tasks is constrained by GPU memory, memory bandwidth, and time. Many inference optimisations exist (weight and KV-cache quantisation, speculative decoding, CPU offloading, and concurrency), but they are usually evaluated in isolation or on short-context workloads. We propose an empirical comparison of these techniques on a fixed GPU and a fixed time budget, using the Qwen3 family and the MTP-native Qwen3.5-9B model evaluated on the MATH-500 benchmark. Our main novelty lies in systematically evaluating these optimization techniques jointly. Our final deliverables will include fully reproducible trade-off analyzes and actionable recommendations for deploying reasoning models on resource-constrained hardware.

1 Introduction

1.1 Scientific goal

Executing open-weight large language models for complex mathematical reasoning requires balancing model intelligence against strict hardware constraints. While individual inference optimization techniques have proven effective for standard benchmarks, their combined behavior under long-context Chain-of-Thought (CoT) generation remains unmapped. In this project, we address the challenge of maximizing task resolution on a standardized mathematical benchmark given a single GPU and a strict wall-clock execution limit. We conduct a controlled empirical comparison across key optimization axes – quantisation, Multi-Token Prediction (MTP), CPU offloading, and concurrency – to identify the optimal deployment configuration.

Research questions.

- **RQ1.** How do weight and KV-cache quantisation, separately and jointly, affect GPU memory usage, mathematical reasoning accuracy, and the number of correct MATH-500 answers within one hour?
- **RQ2.** How do concurrency and CPU offloading affect latency, throughput, and the number of correct answers under fixed GPU and time budgets?
- **RQ3.** How does speculative decoding (draft-model or native MTP) affect decoding speed and the number of correct answers as concurrency increases?

Hypotheses.

- **H1.** Joint weight and KV-cache quantisation will reduce GPU memory usage more than either technique alone, enabling higher concurrency, although aggressive quantisation may reduce mathematical reasoning accuracy.
- **H2.** The number of correct answers per hour will peak at an intermediate concurrency level; CPU offloading will not improve

this score when the model and its KV-cache fit entirely in GPU memory.

- **H3.** Speculative decoding will improve decoding speed at low concurrency, with diminishing speedups as concurrency increases.

1.2 Significance

Modern complex workflows, such as agentic coding or mathematical reasoning, push LLM inference infrastructure to its limits by generating thousands of tokens per Chain-of-Thought (CoT) sequence and quickly exhausting VRAM. Practitioners deploying open-weight models on consumer or workstation GPUs face complex trade-offs between memory footprint, execution latency, and accuracy. Existing literature evaluates optimizations like quantisation, PagedAttention, or speculative decoding mostly in isolation. This work evaluates these inference strategies jointly on generation-heavy mathematical reasoning under a strict single-GPU and 1-hour time budget. While focused on mathematical CoT, the underlying bottlenecks – KV-cache saturation and memory bandwidth limits – are universal. Our findings will yield practical recommendations for maximizing serving throughput on resource-constrained hardware, applicable to both reasoning tasks and broader long-horizon autonomous agents.

2 Related Work

2.1 Experimental setting: hardware, benchmark, harnesses, models

Our evaluation is anchored on a single NVIDIA RTX 6000 GPU (48 GB VRAM) provisioned from the university's Eden HPC cluster, representing a strict memory and compute boundary for local deployment. As our primary open-weight models, we select `Qwen3-14B` and `Qwen3-8B` (Yang and others, 2025) due to their state-of-the-art mathematical reasoning capabilities. To evaluate the impact of Multi-Token Prediction (MTP), we additionally utilise the MTP-native `Qwen3.5-9B` (Qwen, 2026). We rely on the standardized `LM Evaluation Harness` (Gao et al., 2023) to execute the `MATH-500` benchmark (Hendrycks et al., 2021; Lightman et al., 2024). For inference serving, we deploy `vLLM` (Kwon et al., 2023) with PagedAttention to support efficient batching, prefix caching, and quantisation backends. Finally, CPU offloading evaluations will actively monitor host RAM (64 GB) and PCIe Gen4 transfer rates to identify bandwidth bottlenecks.

2.2 Inference-efficiency techniques

Weight quantisation. Weight quantisation reduces model memory and weight-transfer costs by representing parameters at lower precision. GPTQ (Frantar et al., 2023) uses approximate second-order information to compensate for quantisation errors, while AWQ (Lin et al., 2024) uses activation-aware scaling to preserve salient weights. SmoothQuant (Xiao et al., 2023) instead enables W8A8 inference by shifting quantisation difficulty from activations to weights. Although 4-bit quantisation substantially reduces memory usage, its effect on inference speed depends on hardware and kernel support. Moreover, Mekala et al. (2025) show that quantisation can significantly degrade performance on long-context tasks. Thus, lower precision may increase available KV-cache capacity while potentially degrading mathematical reasoning accuracy.

KV-cache quantisation. KV-cache memory grows with sequence length and the number of concurrent requests, making it a major bottleneck in long-context inference. KIVI (Liu et al., 2024) introduces asymmetric 2-bit quantisation, applying per-channel quantisation to keys and per-token quantisation to values. KVQuant (Hooper et al., 2024) further combines pre-RoPE key quantisation, non-uniform quantisation and outlier handling to preserve accuracy at 3-bit precision. These approaches reduce memory requirements and can support longer contexts or higher concurrency, but may introduce dequantisation overhead. We therefore study KV-cache precision as a trade-off between memory savings, throughput, and mathematical reasoning accuracy.

Speculative decoding and MTP. Autoregressive decoding is limited by memory bandwidth, so spare compute can be used to verify several tokens at once. Leviathan et al. (2023) let a small draft model propose a few tokens that the target model checks in a single forward pass; a rejection rule keeps the output distribution unchanged. Gloeckle et al. (2024) train a model with several output heads (multi-token prediction, MTP), and the extra heads can then draft tokens without a second model. The gain usually shrinks at large batch sizes, but Sadhukhan et al. (2025) show that drafting with a compressed KV cache keeps it for long

contexts, as in long reasoning outputs. We therefore treat speculation as a latency–concurrency trade-off.

CPU offloading. Offloading trades GPU memory for transfers over PCIe and slower CPU compute. FlexGen (Sheng et al., 2023) spreads weights, activations and KV cache over GPU, CPU and disk using a searched placement and a block schedule, which makes a 175B model run on a single 16 GB GPU, but only for latency-insensitive batch workloads. InfiniGen (Lee et al., 2024) keeps the KV cache in host memory and prefetches only the entries predicted to matter for the next layer. NEO (Jiang et al., 2025) moves decoding attention and part of the KV cache to the CPU to enlarge the GPU batch without raising latency. Since PCIe and CPU memory bandwidth limit all of these, offloading should pay off mainly when KV-cache memory limits the batch.

Concurrency and batching. Orca (Yu et al., 2022) introduced iteration-level scheduling to improve batching efficiency, while PagedAttention (Kwon et al., 2023) reduced KV-cache memory fragmentation. Sarathi-Serve (Agrawal et al., 2024) further addressed the throughput–latency trade-off through chunked prefill. These methods motivate our concurrency sweep: increasing parallel requests can improve GPU utilisation, but excessive batching may increase latency and memory pressure, reducing the number of mathematical problems solved within the time budget.

Context compaction. Context compression provides another approach to reducing inference costs. LLMLingua (Jiang et al., 2023) compresses input prompts, while observation masking and summarisation (Lindenbauer et al., 2025; Kang et al., 2026) reduce context growth in long-horizon agents. However, MATH-500 primarily involves independent mathematical problems with potentially long generated reasoning rather than accumulated agent observations. We therefore review context compaction as related work but exclude it from our experiments, focusing instead on KV-cache quantisation and other inference-level optimisations.

2.3 Comparative studies

QServe (Lin et al., 2025) demonstrates that joint weight, activation and KV-cache quantisation (W4A8KV4) can improve serving throughput through system-level co-design. However, aggressive compression does not necessarily preserve complex reasoning capabilities: Dong et al. (2025) report severe task-dependent performance losses under 4-bit quantisation. These findings motivate evaluating compression techniques through both system-level throughput and downstream task accuracy. Our study extends this perspective to evaluate how these trade-offs manifest specifically in mathematical reasoning workflows under fixed hardware and strict wall-clock constraints.

2.4 State-of-the-art summary table

Table 5 in Appendix C summarizes key state-of-the-art inference techniques across quantisation, speculative decoding, CPU offloading, concurrency, and context management.

3 Benchmark and Models

Item	MATH-500 Benchmark
Status	READY
Size	500 tasks (a curated test subset of the original MATH dataset (Hendrycks et al., 2021; Lightman et al., 2024))
Label	Correct if the extracted final answer is mathematically equivalent to the reference according to the fixed evaluation procedure.
Class balance	Problems span 7 mathematical subjects and 5 difficulty levels; distributions are not assumed to be uniform.
Licence	MIT
Obtained from	Hugging Face Hub (e.g., `HuggingFaceH4/MATH-500`)

Table 1: Benchmark used for all experiments. The end task is fixed; we add no data of our own.

Each experimental run is evaluated under a strict 1-hour wall-clock time limit across the 500 tasks of the MATH-500 benchmark (more details in Table 1). A task is scored as solved if and only if the model's generated final answer – typically extracted from a `\boxed{}` LaTeX environment – exactly matches the ground truth solution, as automatically parsed by the LM Evaluation Harness. Tasks that remain unfinished or time out when the 1-hour wall-clock limit is reached are conservatively marked as unresolved (incorrect).

In addition to the benchmark, our evaluation tightly controls the choice of reasoning models to ensure a fair comparison across different optimization axes. Table 2 presents LLM models accessibility. These models were chosen as primary targets because an examination of their base and fine-tuned variants on Hugging Face shows that they achieve MATH-500 scores of 68–85%

in evaluations performed without hardware constraints (Siyuc, nd). Detailed asset statements regarding datasets and open-weight models are provided in Appendix A.

Item	Evaluated Models
Status	READY
Selected Models	`Qwen3-14B`, `Qwen3-8B`, `Qwen3.5-9B`
Project Role	`Qwen3-14B/-8B`: Primary targets for quantisation, concurrency, and offloading sweeps. `Qwen3.5-9B`: Specialized target for native Multi-Token Prediction (MTP) ablation.
Licence	Apache 2.0 (permissive open-weight licences for all selected models).
Obtained from	Hugging Face Hub (e.g., `Qwen/Qwen3-14B`, `Qwen/Qwen3.5-9B`).

Table 2: Overview of the open-weight large language models used in the experiments.

4 Concept and Work Plan

4.1 Pipeline

The experimental workflow is outlined in Figure 1.

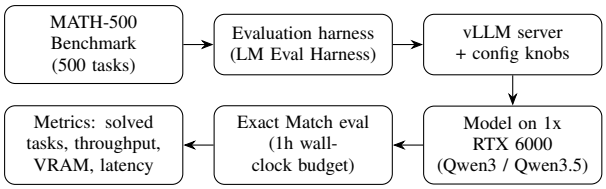


Figure 1: Experimental pipeline. Only the inference-server configuration varies between runs; the MATH-500 benchmark, evaluation harness, target GPU, and 1-hour time budget remain fixed.

4.2 Metrics

Our primary evaluation metric is the total number of tasks resolved within the 1-hour wall-clock limit (out of 500). Secondary system-level metrics include end-to-end generation throughput (tokens/second), per-task latency (median and 95th percentile), time-to-first-token (TTFT), peak GPU VRAM consumption, and average KV-cache capacity utilization. For speculative decoding runs, we record the draft token acceptance rate, while CPU offloading configurations track host RAM usage and PCIe bus saturation. Tasks that fail due to Out-Of-Memory (OOM) errors, exceed maximum context limits, or produce unparsable answers (e.g., missing the `\boxed{}` environment) are logged as unresolved and categorized by failure type.

4.3 Deliverables

- **D1.** Reproducible benchmark setup on fixed hardware with documented harness, model(s), GPU and software versions.
- **D2.** Baseline and best score within the budget, with latency, throughput and memory for every tested configuration.
- **D3.** Recommendation of which techniques give the best trade-offs (e.g. weight quantisation vs. KV-cache space, concurrency vs. per-task latency).

4.4 Work plan

Project timeline is detailed in Table 3. Individual author workloads and task responsibilities are listed in Table 4 in Appendix B.

Period	Task and owner(s)
2–4 Nov	Phase 1: literature review and proposal (all)
5–11 Nov	GPU, vLLM, harness setup; BF16 baseline (Bokhan, Frydrysiak)
12–18 Nov	End-to-end pilot and mid-term video (all)
19 Nov–2 Dec	Baseline analysis, initial sweeps; Phase 2 (all)
3–9 Dec	Weight and KV-cache quantisation (Frydrysiak, Kukla)
10–16 Dec	Concurrency sweep and GPU profiling (Kukla)
17–23 Dec	Speculative decoding and CPU offloading (Pergała)
3–9 Jan	Repeated experiments, joint configurations and error analysis (Bokhan, Pergała)
10–13 Jan	Final report, presentation and README (all)

Table 3: Project schedule for 2026–2027. Winter break: 24 December–2 January.

4.5 Preliminary results and risks

No full experiments have been completed yet. The main risks include timeouts during long CoT generation (mitigated by strict time limits), VRAM exhaustion at high concurrency (addressed via batch limits and quantisation), and stochastic score variation (managed by ≥ 3 repeated runs). Answer parsing failures will be addressed through consistent prompting and robust extraction, while software drift will be controlled by pinned dependencies. The risk of low draft-token acceptance rates in speculative decoding is mitigated by evaluating the natively MTP-enabled Qwen3.5-9B model, while excessive PCIe overhead will be managed by carefully limiting CPU offloading boundaries.

5 Approach and Methodology

(a) Experimental Setting: All evaluations are executed on a single GPU provisioned from the Eden

HPC cluster (NVIDIA RTX 6000, 48 GB VRAM) paired with 64 GB host RAM. We evaluate `Qwen3-14B`, `Qwen3-8B`, and `Qwen3.5-9B` driven by the `LM Evaluation Harness` running exclusively on `vLLM`. All software components are pinned inside a reproducible Docker container.

(b) Baseline Configuration: The default baseline operates in unquantised 16-bit precision (BF16) with standard PagedAttention enabled, a concurrency level of $N = 1$ (sequential task execution), and no offloading or speculative decoding enabled.

(c) One-Factor-at-a-Time Sweeps: We systematically vary one optimization dimension at a time while holding all other parameters fixed:

- *Concurrency & Batching:* Sweeping parallel task counts ($N \in \{1, 2, 4, 8, 16, 32\}$) to observe the trade-off between per-task latency and total 1-hour throughput.
- *Quantisation:* Comparing W4A16 (e.g., AWQ/GPTQ) and 2-bit/4-bit KV-cache quantisation against BF16 baselines.
- *Speculative Decoding & Offloading:* Speculative decoding on/off ($k \in \{3, 5, 7\}$ speculated tokens); offloading on/off, moving as many GB to the CPU as 4-bit quantisation frees (about 75% of the bf16 weights), under a GPU memory cap.

Following individual sweeps, top-performing techniques are combined into joint configurations to evaluate synergistic effects.

(d) Analysis & Error Categorization: We plot score–throughput–latency–memory Pareto frontiers across all configurations. Key settings are run ≥ 3 times to compute confidence intervals. Unresolved tasks are automatically classified into four failure modes: (1) *Timeouts* (exceeding 1 h budget), (2) *Resource Exhaustion* (OOM or context overflow), (3) *Incorrect Answer* (math logic failure), and (4) *Parsing Errors* (missing or malformed `\boxed{}` formatting).

(e) Equipment Specifications: NVIDIA RTX 6000 (48 GB VRAM, PCIe Gen4), 64 GB RAM, Ubuntu 22.04 LTS, Docker v24.0.

Declaration of LLM Use

Gemini Notebook (Google) served as an interactive workspace for Michał Kukla to discover literature, analyze source passages via RAG, generate audio summaries, and accelerate decision-making and LaTeX formatting. All candidate papers were manually vetted for relevance, and all paper selections, technical reasoning, and hypotheses remain the author's original work.

Claude (Anthropic) was used by Sebastian Pergała to search for literature on speculative decoding and CPU offloading, domain knowledge, and to improve the style of the text.

Gemini (Google) was also used by Norbert Frydrysiak and Katsiaryna Bokhan to proofread written sections, format LaTeX tables, and comprehend concepts and ideas from scientific papers.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134.

Peijie Dong, Zhenheng Tang, Xiang Liu, Lujun Li, Xiaowen Chu, and Bo Li. 2025. Can compressed LLMs truly act? an empirical evaluation of agentic capabilities in LLM compression. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 14169–14202.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation, 12.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 15706–15734.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS 2021)*.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems*, volume 37.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376.

Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. NEO: Saving GPU memory crisis with CPU offloading for online LLM inference. In *Proceedings of Machine Learning and Systems*, volume 7.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2026. ACON: Optimizing context compression for long-horizon LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626.

Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286.

Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6.

Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2025. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems*, volume 7.

Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. 2025. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. arXiv:2508.21433; DL4C Workshop at NeurIPS 2025.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344.

Anmol Mekala, Anirudh Atmakuru, Yixiao Song, Marzena Karpinska, and Mohit Iyyer. 2025. Does quantization affect models' performance on long-context tasks? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9422–9470.

Qwen. 2026. Qwen3.5 model family.

Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. 2025. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116.

Siyuc. n.d. INFUSER-rlvr-Qwen3-8B-base. Hugging Face model card. Reports a MATH-500 score of 84.85% for a fine-tuned Qwen3-8B variant.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 38087–38099.

An Yang et al. 2025. Qwen3 technical report. arXiv:2505.09388.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 62557–62583.

A Data and Models Statement

Our benchmark data comprises MATH-500, a publicly available subset of 500 competition-level problems distributed under the MIT Licence and containing no personal data or personally identifiable information. Our model assets comprise three open-weight architectures – `Qwen3-14B`, `Qwen3-8B`, and `Qwen3.5-9B` – accessed in compliance with their permissive Apache 2.0 licences. Both datasets and model weights are retrieved directly from public repositories without modification. All experimental logs generated during our benchmarking contain solely execution telemetry and the models' generated mathematical reasoning.

B Contribution Table

Individual contributions during this phase are listed in Table 4.

Person	Tasks in Phase 1	Hours
Michał Kukla	Concurrency, batching and context compaction: literature review, SOTA comparison, Methodology	15
Katsiaryna Bokhan	Hardware (RTX 6000), serving engine (vLLM) & harness setup, model selection (Qwen3/Qwen3.5), Methodology & Data Statement	16
Sebastian Pergała	Speculative decoding/MTP and CPU offloading: literature review, SOTA comparison, RQ2 & RQ3 and H2 & H3 formulation	13
Norbert Frydrysiak	Weight and KV-cache quantisation literature review, SOTA comparison, RQ1 and H1 formulation	13

Table 4: Contributions and workload in hours.

C Summary of Related Work

Work	Technique	What it does	Data (split)	Reported result	Venue
Lindenbauer et al. (2025)	Compaction	Observation masking vs. LLM summary in SWE-agent	SWE-bench Verified (eval)	Qwen3-Coder 480B: solve rate 53.8% → 54.8%, about half the cost	DL4C @ NeurIPS'25
Kang et al. (2026)	Compaction	Guideline-optimised compression of history and observations	AppWorld (test)	Accuracy 56.0% → 56.5%, peak tokens 9.93k → 7.33k	ICML'26
Li et al. (2025)	Concurrency	KV-cache TTL pinning + program-level scheduling	SWE-bench, BFCL (eval)	1.7× lower job completion time on Llama-3.1 8B/70B	arXiv
Yu et al. (2022)	Batching	Iteration-level scheduling	Serving workloads (N/A)	Up to 36.9× throughput vs FasterTransformer	OSDI'22
Kwon et al. (2023)	Batching / KV	PagedAttention	ShareGPT (N/A)	2–4× throughput improvement over Orca	SOSP'23
Agrawal et al. (2024)	Batching	Chunked prefill	ArXiv, GitHub (N/A)	Up to 5.6× higher throughput at same tail latency	OSDI'24
Zheng et al. (2024)	Prefix reuse	RadixAttention	Chat, Agents (N/A)	Up to 3.1× higher throughput vs vLLM	NeurIPS'24
Frantar et al. (2023)	Weight quant.	Second-order error compensation (GPTQ)	WikiText-2 (OPT-175B, eval.)	PPL: 8.34 (FP16) vs. 8.37 (4-bit)	ICLR'23
Lin et al. (2024)	Weight quant.	Activation-aware scaling	WikiText-2 (Mistral-7B, eval.)	PPL: 4.14 (FP16) vs. 4.30 (INT4-g128)	MLSys'24
Xiao et al. (2023)	W8A8	Smooths activation outliers	LLM inference workloads (N/A)	Up to 1.56× speedup; 2× lower model memory	ICML'23
Liu et al. (2024)	KV quant.	Asymmetric 2-bit K/V compression	LLM inference workloads (N/A)	Up to 2.6× lower peak memory; 2.35–3.47× throughput	ICML'24
Hooper et al. (2024)	KV quant.	Pre-RoPE, non-uniform quant. and outliers	WikiText-2, C4 (eval.)	Under 0.1 PPL degradation at 3 bits	NeurIPS'24
Lin et al. (2025)	Joint quant.	W4A8KV4 system co-design	Serving: 1K input, 512 output (N/A)	Qwen1.5-72B on L40S: 3.5× throughput vs. TensorRT-LLM	MLSys'25
Leviathan et al. (2023)	Speculative decoding	Small draft model, parallel verification; output unchanged	WMT EnDe, CNN/DM (N/A)	T5-XXL, batch 1: up to 3.4× faster	ICML'23
Gloeckle et al. (2024)	MTP	Extra output heads draft tokens without a second model	Code sequences (N/A)	7B, 4 heads: 3.0× faster decoding on code	ICML'24
Sadhukhan et al. (2025)	Speculative decoding	Draft with compressed KV cache for long contexts	RULER word extraction (N/A)	LLaMA-3.1-8B, 100k tokens, batch 41: 2.51× faster	ICLR'25
Sheng et al. (2023)	Offloading	Weights and KV cache over GPU, CPU, disk; searched placement	Synthetic prompts, 512 in/32 out (N/A)	OPT-175B, one 16 GB GPU: 0.69 tokens/s vs. 0.01 for baselines	ICML'23
Lee et al. (2024)	KV offloading	KV cache on CPU; prefetch entries predicted important	PG-19 sentences (N/A)	OPT-13B, batch 20: 1.63–32.93× faster than baselines	OSDI'24
Jiang et al. (2025)	Offloading	Decoding attention and part of KV cache on CPU	Synthetic workloads (N/A)	vs. GPU only: +14% (H100), +26% (A10G), up to 7.5× (T4)	MLSys'25

Table 5: Summary of related work. "Data (split)" gives the dataset and evaluation split of each reported number; numbers from different hardware or benchmarks are not directly comparable.